Architetture e sistemi operativi

Contents

1 Introduzione **4**

2 Processore, memoria e istruzioni **5**
2.1 Processore . . . 5
2.2 Memoria . . . 5

3 Livelli di astrazione dei calcolatori **6**
3.1 Principi importanti . . . 6

4 Circuiti Digitali **8**
4.1 Sistema binario . . . 8
4.2 Numeri negativi . . . 8
4.3 Numeri con la virgola (Floating Point) . . . 9
4.4 Caratteri (ASCII) . . . 9
4.5 Notazione per grandezza input/output . . . 10
4.6 Espressioni dei circuiti . . . 10
4.7 Tempi per calcolare risultati . . . 10
4.8 Esadecimale . . . 11
4.9 Processo di progettazione dei circuiti digitali . . . 12
4.10 Mappe di Karnaugh . . . 13
4.11 Disturbi aleatori (ALEA) . . . 14

5 Verilog **15**
5.1 Dichiarazione moduli . . . 15
5.2 Modulo tester . . . 16
5.3 Monitor . . . 16
5.4 Full Adder . . . 16
5.5 Tools . . . 16

6 Moduli **17**
6.1 Sommatore . . . 17
6.2 Comparatore . . . 17
6.3 Codificatore . . . 18
6.4 Decoder . . . 18
6.5 Moduli per algebra booleana . . . 18
6.6 Full Adder . . . 19
6.6.1 Full Adder in complemento a 2 . . . 20
6.7 Multiplexer . . . 21
6.8 Demultiplexer . . . 21

7 Memoria **23**
7.1 Registri . . . 23
7.2 SR Latch . . . 23
7.3 D Latch . . . 24
7.4 Flip Flop D . . . 25
7.5 Segnale di reset . . . 26

7.6 Reti sequenziali . . . 26
7.7 RAM . . . 28
7.7.1 RAM Dinamica . . . 28
7.7.2 RAM Statica . . . 29
7.8 ROM . . . 30
7.9 Memorie associative . . . 31
7.10 Memorie modulari . . . 31
7.10.1 Organizzazione sequenziale . . . 32
7.10.2 Organizzazione interlacciata . . . 33

8 Assembler **34**
8.1 Definizione assembler . . . 34
8.2 PC e il suo valore . . . 34
8.3 Parola di stato (CPSR) . . . 34
8.4 Codice assembler (Assembly) . . . 35
8.4.1 Introduzione . . . 35
8.4.2 Schema interno . . . 35
8.4.3 Convenzioni nell'utilizzo dei registri . . . 36
8.4.4 Sommario dei comandi . . . 36
8.4.5 ADD . . . 37
8.4.6 LSR e LSL . . . 37
8.4.7 CMP . . . 37
8.4.8 Salti . . . 37
8.4.9 LDR . . . 38
8.4.10 Loop . . . 38
8.4.11 Operazioni condizionali . . . 39
8.4.12 Comandi che settano flag . . . 39
8.4.13 Prelevare registri dallo stack . . . 39

9 Altro **41**

10 Esercizi **42**
10.1 Cosa calcola una rete . . . 42
10.1.1 Descrizione . . . 42
10.1.2 Soluzione . . . 42
10.2 Circuito di una sveglia . . . 43
10.2.1 Descrizione . . . 43
10.2.2 Soluzione . . . 43

1 Introduzione

In questo corso andremo a:

- Capire come funziona un elaboratore.
- Capire quali tipi di istruzioni è in grado di eseguire un elaboratore nativamente.
- Come lo si può dotare di un sistema operativo.

Il corso utilizza Windows e Linux come esempi di sistemi operativi.

Codice Teams: 6uk5ntg

Email Docente: marco.delutto@unipi.

2 Processore, memoria e istruzioni

2.1 Processore

Il **processore** è in grado di fare operazioni piccole.
Il processore contiene i **registri**, memorie piccolissime e velocissime utilizzate per memorizzare dati temporanei legati alle istruzioni da eseguire. Ecco alcuni registri speciali:

- **Program Counter (PC)**: un registro speciale che conserva lo stato di esecuzione memorizzando l'indirizzo della prossima istruzione da eseguire.
- **Instruction Register (IR)**: contiene l'istruzione prelevata dalla memoria che deve essere eseguita.
- **Stack Pointer (SP)**: contiene la posizione attuale dello stack.

2.2 Memoria

Il processore è unito ad una **memoria**, costituita da **celle**, è vista come un vettore:

$$M[i] = k$$

Ogni cella ha un valore e la sua capacità è 1 parola (32 o 64 bit).
Una volta che il processore viene acceso, esso esegue:

```
while(true):
    instr = getInstructionFrom(M)
    decoded = decode(instr)
    execute(decoded)
    determineResults()
    ...
```

Il processore quindi:

- Estrae istruzioni dalla memoria
- Le decodifica
- Le esegue
- Calcola i risultati

3 Livelli di astrazione dei calcolatori

Per capire come opera un computer osserviamo dei livelli di astrazione, sviluppati a "torre" nel seguente modo:

applicazioni
sistema operativo
architettura (set istruzioni)
μ - architettura (processore)
logica (componenti)
circuiti digitali (0/1, 5V/0V)
circuiti analogici
dispositivi
fisica

Non andremo a vedere niente delle caratteristiche hardware degli ultimi 3 livelli, non ci interessa.
Per capire:

- Noi partiamo col realizzare dei circuiti digitali che servono per creare dei componenti
- Tali componenti ci servono per realizzare un processore
- Al processore dovremo dare in pasto delle istruzioni
- Le istruzioni servono al sistema operativo e alle applicazioni

Tra i livelli di "architettura" e "sistema operativo" si trova un **interfaccia**, genericamente attribuita al livello **i**. Questo significa che:

- Il livello del sistema operativo corrisponde al livello **i** (si trova al livello dell'interfaccia)
- Il livello dell'architettura corrisponde al livello **i - 1**

Al livello **i + 1**, tra sistema operativo e applicazioni, si trovano le **chiamate al sistema operativo**, chiamate **API**.

Ogni livello conosce solo il **livello inferiore** a se, questo è per mantenere i livelli superiori attraverso diversi livelli inferiori; per esempio un telefono Samsung e un Motorola sono realizzati in maniera diversa, ma entrambi utilizzano Android, questo è possibile proprio grazie al fatto che i livelli non si concentrano sui livelli soprastanti.
Un altro esempio: non si possono portare app iOS su Android.
Spesso esistono dei traduttori che permettono di convertire istruzioni (x64,x86) per permettere compatibilità con sistemi operativi e processori differenti.

3.1 Principi importanti

I componenti che svolgono le istruzioni seguono questi concetti:

- **Gerarchia**: il principio dei livelli, ogni livello utilizza i componenti dei livelli sottostanti senza occuparsi di cosa fanno.

- **Modularità**: presenza di moduli distinti per la specializzazione e composizione del lavoro.
- **Regolarità**: si realizzano strutture simili e ripetibili invece di fare sempre le cose in modo diverso.

Si usano moduli più piccoli per svolgere azioni più grandi quando combinati.

4 Circuiti Digitali

4.1 Sistema binario

Noi utilizziamo il sistema decimale, ovvero un sistema **posizionale**, dove ogni cifra possiede un valore diverso a seconda della sua posizione.
Anche il **sistema binario** è un sistema posizionale, ma le cifre sono solo 2 (0,1).

- **Esempio decimale:**
$$13_{10} = (1 \cdot 10^1) + (3 \cdot 10^0)$$
- **Esempio binario:**
$$10_2 = (1 \cdot 2^1) + (0 \cdot 2^0) = 2_{10}$$

Nei computer si utilizza il sistema binario per via della sua semplicità e della sua compatibilità con i moduli logici, per esempio una moltiplicazione è più facile da svolgere in binario tramite un modulo **AND**, invece che in decimale.

4.2 Numeri negativi

Per rappresentare i numeri negativi in binario ci sono due modi:

- **Modulo e segno:** si riserva il **MSB** (Most Significant Bit, più a sinistra) per il segno (1 se negativo, 0 se positivo).
- **Complemento a 2:** nato perche complicate le operazioni con modulo e segno (if else); si ha un algoritmo per ottenere il binario di un numero negativo:
  1. Si trasforma il numero in binario.
  2. Si fa il NOT di ogni bit
  3. si aggiunge "+1"

Su un byte di solito si possono realizzare **256** possibili combinazioni positive, ma col complemento a 2 l'intervallo si spezza:

- 128 valori positivi (0 - 127)
- 128 valori negativi (-128 - -1)

Nel modulo e segno addirittura ci sono 2 rappresentazioni diverse per lo 0, altro motivo per cui è stato introdotto il complemento a 2.
Ecco come funziona veramente il complemento a 2:

- Il MSB vale un valore negativo (-2^7)
- Tutti gli altri bit valgono valori positivi
- Se si ricalcola la somma di un complemento a 2 per trovare il suo valore decimale, si ottiene il numero negativo.

Per esempio, calcoliamo -10:

$$10_{10} = 00001010_2$$

$$!(00001010) = 11110101$$

$$11110101 + 1 = 11110110$$

$$11110110_2 = (-2^7 + 2^6 + 2^5 + 2^4 + 2^2 + 2)_{10} = (-128 + 64 + 32 + 16 + 4 + 2) = -10_{10}$$

ATTENZIONE: se si effettua una somma tra numeri in complemento a 2 che hanno segni concordi e si ottiene un risultato con segno diverso allora significa che c'è stato un **overflow**.

4.3 Numeri con la virgola (Floating Point)

Il **floating point (FP)** permette di rappresentare i numeri con cifre decimali.
Ecco la sua struttura:

[segno][esponente][mantissa]

Questa struttura può essere lunga 32 o 64 bit.

- **Segno**: 1 bit, segno (+/-) del numero.
- **Esponente**: 8 o 11 bit, esponente per cui "10" è moltiplicato il numero decimale, per esempio:
$$10,5 = 105 \cdot 10^(- 1)$$
- **Mantissa**: 23 o 52 bit, contiene il numero intero da moltiplicare al "10" ().

4.4 Caratteri (ASCII)

Per rappresentare i caratteri in binario si utilizza la **codifica ASCII**, che utilizza **ottetti** (7/8 bit) per rappresentare ogni carattere.
Ogni carattere è identificato da un **ID**. Offre:

- 26 lettere maiuscole
- 26 lettere minuscole
- 10 cifre
- Punteggiatura
- Caratteri speciali (invio, tab, DEL)

Vediamo i range di ID che ci interessano:

- **Non stampabili:** $0 - 31, 127$
- **Stampabili:** $32 - 126$
- **Cifre:** $48 - 57$
- **Lettere maiuscole:** $65 - 90$

- **Lettere minuscole:** $97 - 122$
- **Spazio:** 32

Come si può notare, ogni lettera maiuscola è sempre alla stessa distanza dalla sua corrispondente minuscola: per questo basta aggiungere "**32**" ad una lettera maiuscola (o sottrarlo alla minuscola) per switchare tra lowercase e uppercase.

4.5 Notazione per grandezza input/output

Per indicare quanti bit è grande un input o output, si usa questa notazione:

$$n$$

$$\nrightarrow$$

4.6 Espressioni dei circuiti

Per rappresentare i circuiti si usano delle **espressioni**, vediamo delle notazioni di algebra booleana in tale progettazione:

- Quando un parametro ha un "overline" come $\overline{x}$ allora significa che passa da un NOT.
- La moltiplicazione tra variabili corrisponde all'AND, dato che se facciamo la moltiplicazione tra più valori booleani il risultato sarà diverso da 0 solo se tutti i membri sono settati a "1".
- La somma tra variabili corrisponde all'OR, dato che il risultato è non nullo solo se almeno uno dei membri è settato a "1".

Nota bene: un qualsiasi valore positivo, anche se diverso da "1", viene contato come **true**. Vediamo un esempio:

$$\overline{x}yz + x\overline{y}z + xy\overline{z} + xyz$$

Questo è un circuito con 4 moduli AND, 3 moduli OR e 3 moduli NOT realizzato in questo modo:

4.7 Tempi per calcolare risultati

Quando realizziamo circuiti digitali bisogna tenere di conto quanto ci mettono le porte a inizializzarsi e quanto ci mette il risultato a essere emesso.
Indichiamo il ritardo di una porta da massimo **8 ingressi** con:

$$\Delta t$$

Si considera il ritardo per le porte di AND e OR, il tempo della porta NOT è talmente insignificante che non viene considerato ($= 0$).
Nei nostri circuiti si usano sempre porte da 8.
Per realizzare porte da 9 o più è necessario collegare più porte insieme e farle congiungere ad una porta unica.
Quando bisogna realizzare circuiti con tanti ingressi si tende a realizzare una struttura ad albero.
Ecco come si trova il tempo di impiego di un modulo:

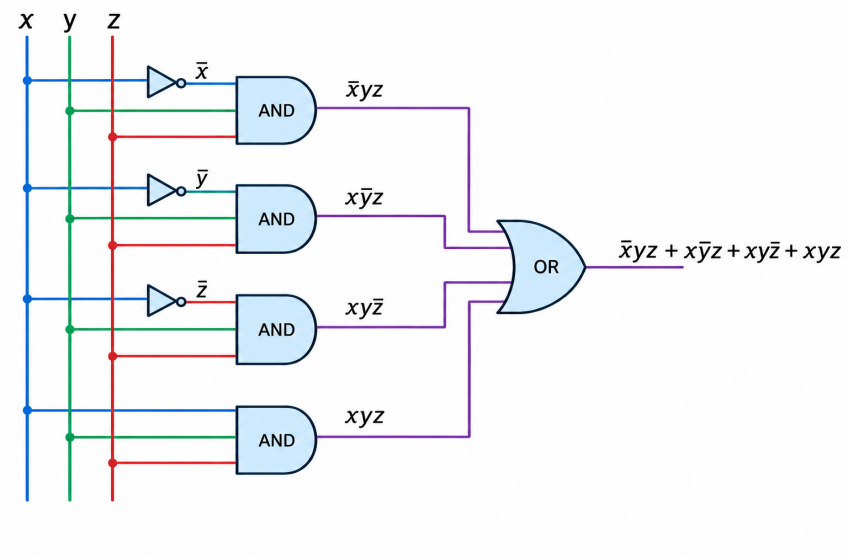


Figure 1: Circuito logico

1. Si crea la tabella di verità con a sinistra gli input e a destra gli output
2. Si scrivono tutte le righe possibili degli input (combinazioni, 2^n, n = numero di input diversi)
3. Si calcola il risultato a seconda del modulo e si scrive per ogni combinazione
4. Ripetere l'istruzione precedente per ogni output
5. Una volta trovata la tabella completa, si trova l'espressione per calcolare ogni output: si eguaglia un output alla somma dei prodotti dei vari input delle combinazioni che hanno risultato un output = "1" (espressione logica, i bit = 0 sono "negati", ovvero con la overline (NOT))
6. Si contano termini (OR) e variabili (AND), spesso ogni modulo li usa entrambi, arrivando al minimo a $2\Delta t$, però se uno tra i due supera il numero 8 allora si conta un tempo in più.

Spesso le formule si possono semplificare, ma ciò non influisce il tempo impiegato dai moduli, bensì ci può permettere di risparmiare materiale e spazio.

4.8 Esadecimale

Oltre al sistema binario, spesso viene utilizzato anche il sistema **esadecimale**, ovvero in base 16.
Questo sistema è comodo, basta pensare al fatto che 16 in binario necessita di esattamente 4 bit (un **nibble**), questo vuol dire che ogni 4 bit si ha una cifra in esadecimale e viceversa.
Ecco le cifre:

$$1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F$$

Con:

$$A = 10, F = 16$$

La base esadecimale viene introdotta da un "**0x**", così:

$$0x5AF$$

4.9 Processo di progettazione dei circuiti digitali

Ecco il procedimento:

1. Descrizione a parole della richiesta
2. Realizzazione pseudo codice
3. Approccio strutturale, composizione componenti
4. Tabella di verità
5. Somma di prodotti (espressione)
6. Rete di porte
7. Tempo di stabilizzazione

Esempio : dato x da n bit:

```
if(pari(x)) {
    x++;
} else {
    x--;
}
```

Per trovare i pari basta notare che tutti i numeri pari in binario hanno il LSB (Least Significant Bit) segnato a "0".

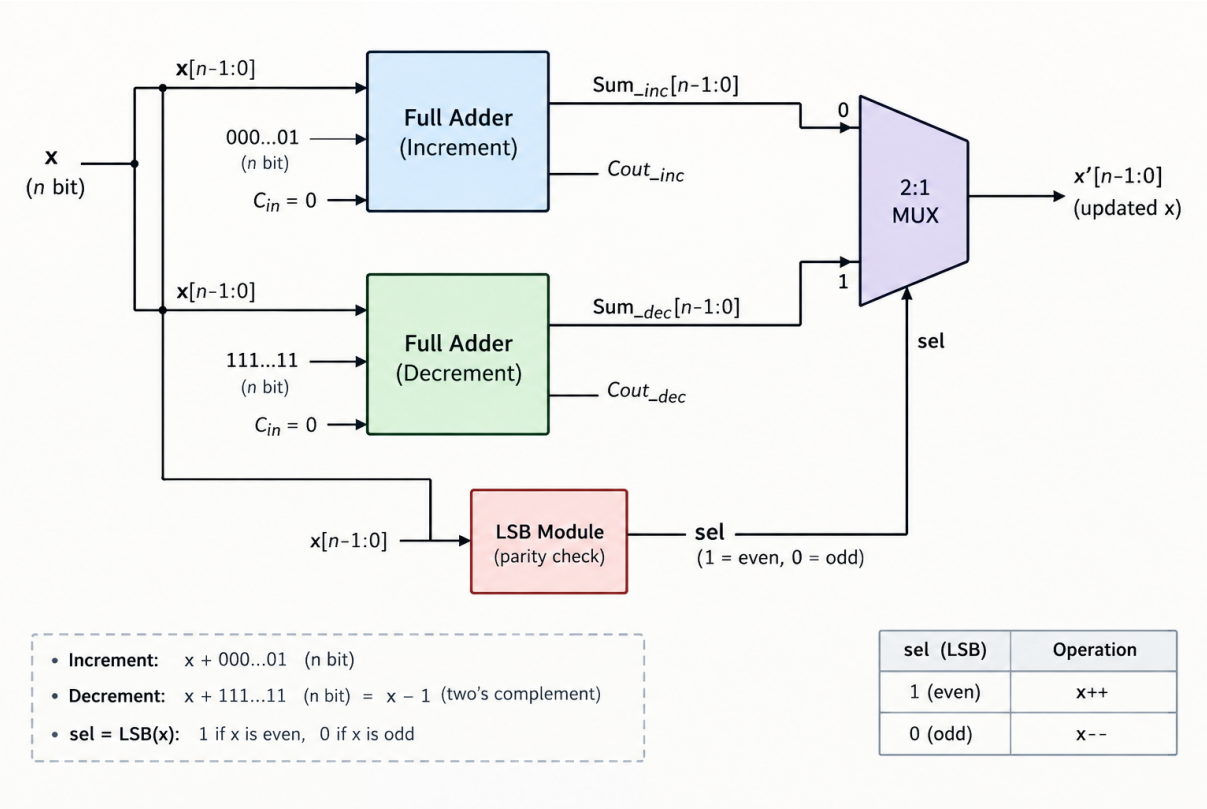


Figure 2: Circuito Pari-Dispari

4.10 Mappe di Karnaugh

Modo alternativo alle tabelle di verità, permette di semplificare notevolemnte le espressioni logiche, si rappresentano cosi:

	$x = 0$	$x = 1$
$y = 0$	0	0
$y = 1$	0	0

	$x_1x_0 = 00$	$x_1x_0 = 01$	$x_1x_0 = 11$	$x_1x_0 = 10$
$y_1y_0 = 00$	0	0	0	0
$y_1y_0 = 01$	0	0	0	0
$y_1y_0 = 11$	0	0	0	0
$y_1y_0 = 10$	0	0	0	0

Come si può vedere viene prima "11" che di "10" perche si usa il **codice Gray**, il quale indica che ad ogni passaggio (celle adiacenti) può cambiare un solo bit.

Una volta che abbiamo fatto la tabella, si **raggruppano gli 1** adiacenti che formano un gruppo di grandezza pari ad una potenza di 2, scegliendo quello più grande (aiuta a ridurre di più).
Per esempio:

	$x_1x_0 = 00$	$x_1x_0 = 01$	$x_1x_0 = 11$	$x_1x_0 = 10$
$y_1y_0 = 00$	1	1	0	0
$y_1y_0 = 01$	1	1	0	0
$y_1y_0 = 11$	0	0	0	0
$y_1y_0 = 10$	0	0	0	0

L'espressione in questo caso sarebbe:

$$F = \overline{x_1}\,\overline{x_0}\,\overline{y_1}\,\overline{y_0} + \overline{x_1}\,x_0\,\overline{y_1}\,\overline{y_0} + \overline{x_1}\,\overline{x_0}\,\overline{y_1}\,y_0 + \overline{x_1}\,x_0\,\overline{y_1}\,y_0$$

Le prime 4 celle possono essere raggruppate.

ATTENZIONE: anche le celle opposte sono considerate adiacenti, come se la tabella fosse arrotolata.
Una volta trovato il gruppo, costruiamo l'espressione semplificata seguendo queste regole:

- Se nell'analisi degli "1" una variabile **cambia**, allora viene **esclusa**.
- Se invece rimane costante, viene **tenuta**.

Nell'esempio precedente:

- x_1 non cambia, rimane.
- x_0 cambia, viene scartata.
- y_1 non cambia, rimane.
- y_0 cambia, viene scartata.

L'espressione adesso è quindi:

$$F = \overline{y_1 x_1}$$

4.11 Disturbi aleatori (ALEA)

Quando si effettuano le transizioni da un valore all'altro tramite delle porte logiche, le quali hanno dei ritardi, possono crearsi delle variazioni indesiderate temporanee, ovvero sbalzi di valore spontanei che possono provocare glitch.
Per esempio, se abbiamo:

$$F = AB + \overline{A}C$$

Se il valore di A cambia da 0 a 1, il ritardo delle porte potrebbe non propagare il cambiamento di valore al secondo modulo dell'espressione in tempo, generando un calcolo errato.
Per risolverli, si modifica la rete logica e si introducono termini ridondanti, i quali nella mappa di Kernaugh "allacciano" insiemi separati.
Più formalmente, ogni coppia di gruppi adiacenti nella tabella di Kernaugh devono essere collegati da un terzo gruppo, come un "ponte".

Applicare queste modifiche di rete sicuramente aumenta la complessità e il tempo impiegato, ma in questo modo ci assicuriamo che la rete funzioni senza errori.

5 Verilog

Verilog è un **HLD (Hardware Descritpion Language)**, ovvero un linguaggio di programmazione che descrive i circuiti logici.
Utilizzeremo:

- iverilog: compilatore Verilog.
- vvn: interprete (genera dumptest).
- gtkwave: mostra le onde grafiche (GUI).

Questi comandi vanno utilizzati in sequenza su un file Verilog per testarlo ed eseguirlo.

5.1 Dichiarazione moduli

Si ha in input uno schema di un circuito scritto con Verilog (.v), il quale viene testato da un altro file / funzione "test"(.v).
Viene generato un file di estensione ".vcd" che contiene i risultati.
Ogni pezzo di circuito deve essere rappresentato come **modulo**, vediamo come creare un MUX:

```
primitive mux(output z, input x, input y, input c);
table
    1 ? 0 : 1;
    ? 1 1 : 1;
    0 ? 0 : 0;
    ? 0 1 : 0;
endtable
endprimitive
```

Per convenzione, si mettono prima gli output e poi gli input.
Da standard, input e output specificati nel modo soprastante sono di **1 bit**.
Dopo è stata specificata una **tabella**, la quale corrisponde proprio alla tabella di verità (sintetizzata in questo caso). I "?" indicano un valore qualsiasi.
Giustamente, la tabella e il primitive vanno chiusi.
Primitive però non si usa mai, si usa **module**:

```
module mux (...)
```

Per calcolare l'**espressione logica** si dichiara un modulo con **module** e poi si fa cosi:

```
module mux(output z, input a, input b, input c);
assign
    z = b && c || a && !c
```

Si può utilizzare il bit di controllo per scegliere l'output:

```
module mux(output z, input a, input b, input c);
assign
    z = (c == 0 ? a : b)
```

5.2 Modulo tester

Ecco come si testa (esegue) il circuito progettato:

```
(circuito)
\dots
module test():

    reg ix, iy, ic;
    wire out;

    mux istanzaMux(out, ix, iy, ic);

    integer i;

    initial
        begin
            $dumpfile("test.vcd");
            $dumpvars
            for(i=0;i<8;i=i+1)
```

Si apre la funzione "test" che contiene le istruzioni del run-time.
Si istanziano le variabili necessarie e i moduli necessari.

5.3 Monitor

Questo comando restituisce i risultati specificati ogni volta che viene passato un input e stampa un messaggio custom:

```
(tester)
\dots
$monitor($time," in = %b %b controllo = %b ==> z = %b", ina, inb, inc,
...
```

5.4 Full Adder

```
module far(output c, output z, input x, input y, input r);
    far ris(z,x,y,r)
    far car(c,x,y,r)
```

5.5 Tools

Ecco alcuni strumenti importanti per lavorare con Verilog:

- **Quartus Prime Lite Edition:** rappresentazione grafica dei moduli utilizzati in un file Verilog.

6 Moduli

Questa sezione elenca vari esempi di moduli per le operazioni del calcolatore.

6.1 Sommatore

Semplice modulo che:

- Prende in input due numeri (1 bit) ed un riporto iniziale di "0" (non lo considera).
- Li somma e restituisce il risultato in output.
- Se c'è overflow, setta a "1" il secondo output che indica il riporto.

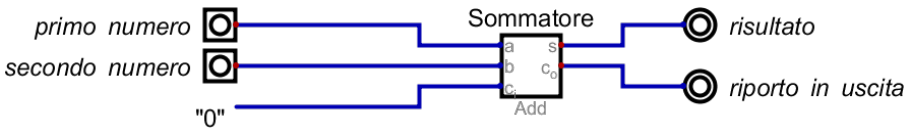


Figure 3: Sommatore

Un sommatore a più bit è costruito nel solito modo.

6.2 Comparatore

Modulo che prende due numeri in input (a, b) e restituise 1 output:

$$out = \begin{cases} 1, a == b \\ 0, a\ != b \end{cases}$$

A	B	Out
0	0	1
0	1	0
1	0	0
1	1	1

Table 1: Tabella di verità del comparatore

Formula (con XNOR):

$$A \odot B$$

Si trasforma in:

$$(A \cdot B) + (\overline{A} + \overline{B})$$

Tempo d'esecuzione: $2\Delta t \cdot N$ con N = numero di bit dei numeri, perche si mettono tanti di questi in parallelo collegati finalmente da un AND con N porte.

6.3 Codificatore

Ha un input e un output, restituisce la posizione dell'1 nell'input. Denotato da una scatola con "COD" scritto dentro.

6.4 Decoder

Un decoder prende in input un numero e attiva una porta in uscita in base a tale ingresso.

Tempo d'esecuzione: Δt (1 solo AND per ogni output)

6.5 Moduli per algebra booleana

Per svolgere i controlli booleani si utilizza una combinazione (circuito) dei seguenti moduli:

- Modulo NOT



Figure 4: Modulo NOT

- Modulo AND

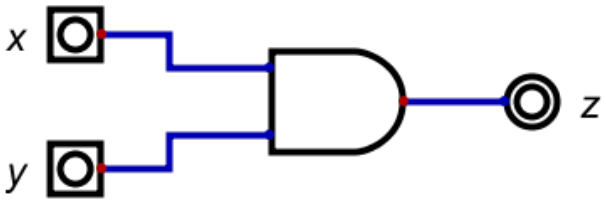


Figure 5: Modulo AND

- Modulo NAND

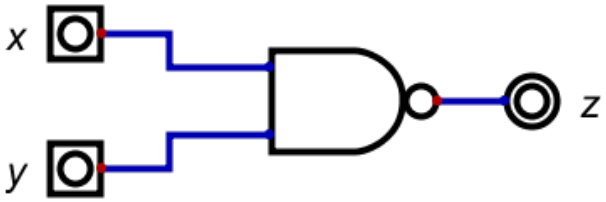


Figure 6: Modulo NAND

- Modulo OR

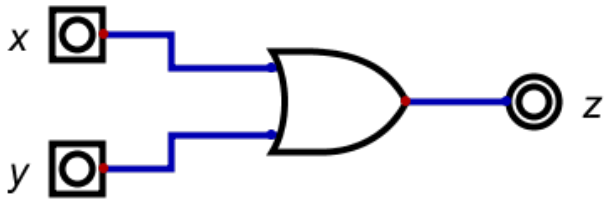


Figure 7: Modulo OR

- Modulo NOR

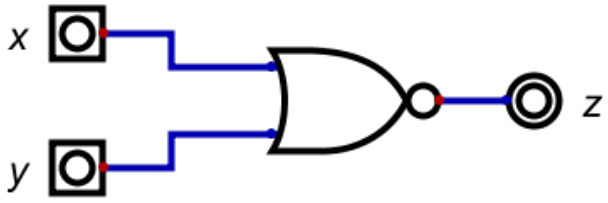


Figure 8: Modulo NOR

- Modulo XOR

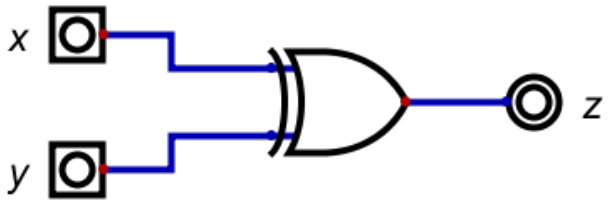


Figure 9: Modulo XOR

- Modulo XNOR

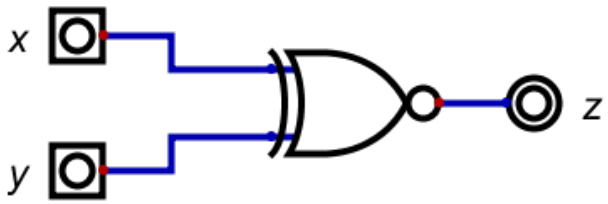


Figure 10: Modulo XNOR

Tutti prendono due o più input in entrata e restituiscono un output ad eccezione del NOT, che prende un solo input.
Inoltre, tutti questi (tranne il NOT che per convenzione non viene considerato) hanno un ritardo di $1\Delta t$.

6.6 Full Adder

Sommatore speciale che:

- Ha 2 input

- Ha 1 output risultato
- Ha 1 output di riporto (che va al prossimo FA, messi in sequenza per sommare piu numeri)
- Ha 1 riporto iniziale (anche nullo)

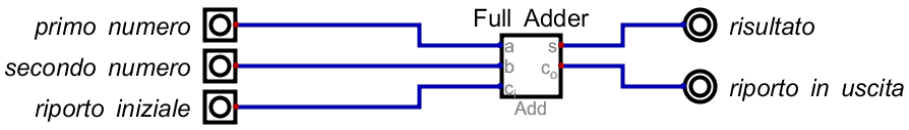


Figure 11: Full Adder

X	Y	C_{in}	Ris	Rip
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Table 2: Tabella di verità del Full Adder

$$Ris = X \cdot \overline{Y} \cdot \overline{C_{in}} + \overline{X} \cdot Y \cdot \overline{C_{in}} + \overline{X} \cdot \overline{Y} \cdot C_{in} + X \cdot Y \cdot C_{in}$$

$$Rip = X \cdot Y + X \cdot C_{in} + Y \cdot C_{in}$$

Un Full Adder a N consiste in più FA messi in sequenza che sommano a coppie di bit alla volta e portano il riporto in uscita al riporto in ingresso del prossimo FA.

Tempo d'esecuzione: $2\Delta t \cdot N$ con N = numero di bit dei numeri.

6.6.1 Full Adder in complemento a 2

Un Full Adder può sommare numeri in complemento a 2 (negativi) con queste particolari modifiche:

- L'input da portare in complemento a 2 viene negato
- Il riporto iniziale è necessariamente uguale a 1

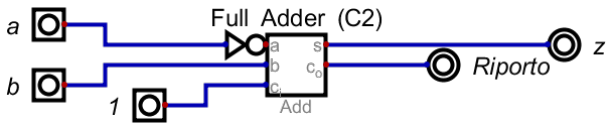


Figure 12: Full Adder (C2)

Tempo d'esecuzione: $2\Delta t \cdot N + \Delta t$ con N = numero di bit dei numeri.

6.7 Multiplexer

Il **multiplexer** è un modulo molto importante che sceglie uno tra due input in base ad un input di controllo che indica la porta dell'input da restituire.

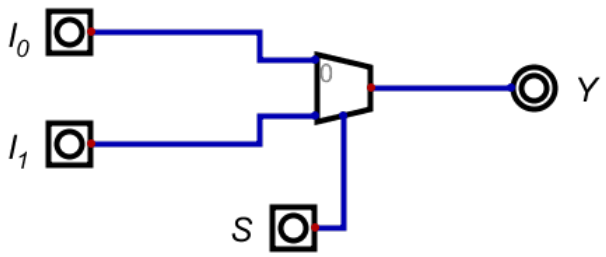


Figure 13: Multiplexer

I_0	I_1	S	Y
0	0	0	0
0	1	0	0
1	0	0	1
1	1	0	1
0	0	1	0
0	1	1	1
1	0	1	0
1	1	1	1

Table 3: Tabella di verità del Multiplexer 2:1

$$Y = \overline{S} \cdot I_0 + S \cdot I_1$$

In realtà il multiplexer può avere più input, ma questo prevede l'espansione della grandezza dell'input di controllo.

Tempo d'esecuzione: $2\Delta t$ se usiamo MUX 2:1 e li mettiamo paralleli, $2\Delta t \cdot \log_8 N$ se usiamo MUX con ingressi multipli.

6.8 Demultiplexer

Contrario del multiplexer, ha 1 input e 2 porte in uscita.
Trasmette l'input in una porta in uscita a seconda del **bit di controllo** in input e azzera la porta in uscita non utilizzata.

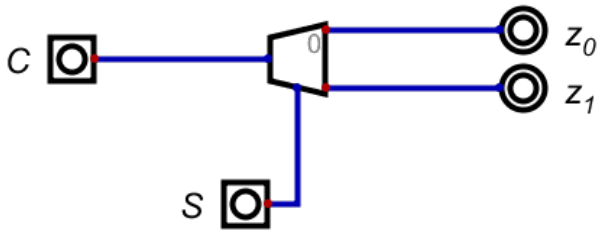


Figure 14: Demultiplexer

C	S	z_1	z_0
0	0	0	0
0	1	1	0
1	0	0	0
1	1	0	1

Table 4: Tabella di verità del Demultiplexer 1:2

$$z_1 = C \cdot \overline{S}$$

$$z_0 = C \cdot S$$

Tempo d'esecuzione: Δt (fa solo AND).
Anche questo, come il multiplexer, può avere più porte in uscita, ma prevedono appunto l'espansione dell'input di controllo.

7 Memoria

7.1 Registri

Il registro funziona cosi:

- Input x di n bit
- Output y di n bit
- Clock in input
- Reset in input (Per quando bisogna fare il RESET) (non presente in figura)
- **Write Enable** in input (dice se scrivere o meno)

$$\begin{cases} clock == 1 \&\& we = 1 \implies \text{ricordati } x \\ \text{altrimenti ricordati il valore precedente} \end{cases}$$

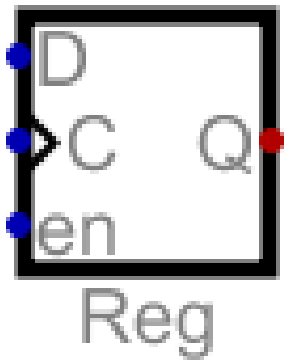


Figure 15: Registro

7.2 SR Latch

Abbiamo un modulo che ha:

- Input **SET**
- Input **RESET**
- Write Enable in input (Non presente in figura)
- Output su 1 porta tra 2 (Q)

$$\begin{cases} SET == 1 \implies \text{ricordati } 1 \\ RESET == 1 \implies \text{ricordati } 0 \end{cases}$$

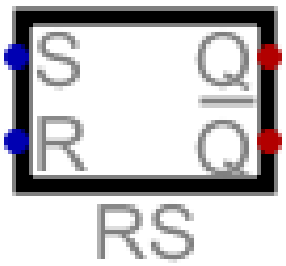


Figure 16: SR Latch

Per realizzare questa cosa bisogna:

- Collegare una porta OR Negata a ogni input (1 ciascuno)
- L'output di ognuno ritorna alla porta OR Negata come secondo input di tale porta dell'altro.

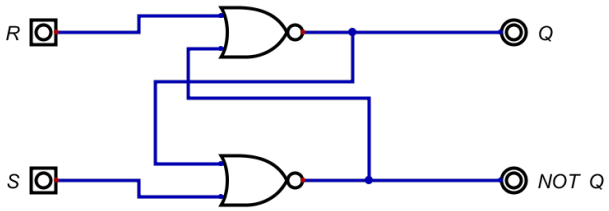


Figure 17: Circuito SR Latch

Ecco la tabella di verità:

S	R	Q	$\overline{Q}$	Stato
0	0	$Q(t)$	$\overline{Q}(t)$	Mantiene
0	1	0	1	Reset
1	0	1	0	Set
1	1	0	0	Vietato

Table 5: Tabella di verità del latch SR con porte NOR

Si crea un problema se entrambi SET e RESET sono settati a 1 perche si crea questo:

$$Q = 0 = \overline{Q}$$

7.3 D Latch

Per risolvere il problema di SR latch, si introducono due input:

- **EN**: dice quando scrivere (sensibile al livello), talvolta viene scritto **CLK**.
- **D**: dice cosa scrivere.

Ecco lo schema interno di tale modulo:

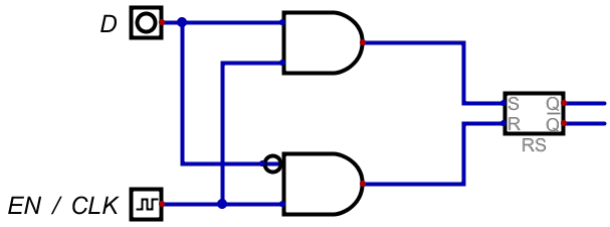


Figure 18: Circuito interno di D Latch

Di solito però viene rappresentato in questo modo:

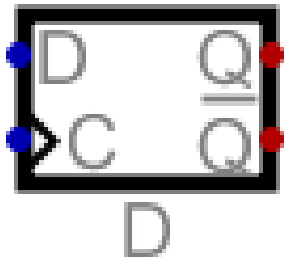


Figure 19: D Latch

Sembra simile all'SR Latch, ma utilizzando un paio di questi possiamo realizzare un **Flip Flop D**.

7.4 Flip Flop D

Un **Flip Flop D** può essere realizzato mettendo in sequenza due D Latch in questo modo:

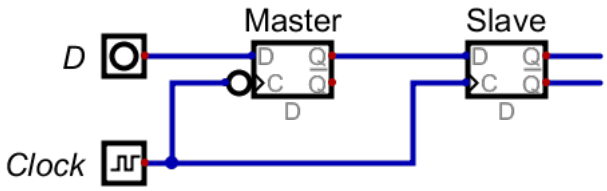


Figure 20: Flip Flop D: modello Master-Slave

Il primo D Latch si chiama **master** (prepara il dato) e il secondo **slave** (trasferisce all'uscita).
I due D Latch sono sensibili al livello, ad uno stato scrive il master, all'altro scrive lo slave.
Tutto il circuito intero, ovvero il **flip flop**, è invece sensibile al fronte del clock, ovvero quando si deve effettuare il cambio tra master e slave.
Se vogliamo, possiamo includere un Write Enable che gestisce i tempi di attivazione:

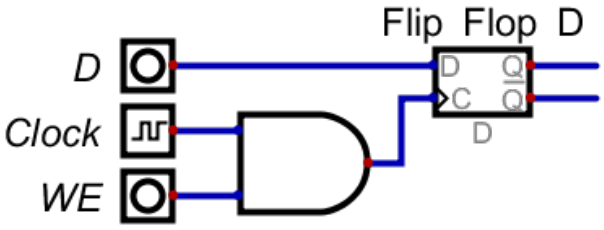


Figure 21: Flip Flop D: Write Enable integrato

7.5 Segnale di reset

Per riportare a "0" un valore di un registro si usa un segnale di **reset**.
Esso viene portato in input in un registro.
La sua versione sincrona è dipendente dal fronte del clock.

7.6 Reti sequenziali

Con i registri si possono sviluppare le **reti sequenziali** per rappresentare **automi**.
Supponiamo di avere una sequenza di bit qualsiasi, bisogna svolgere una sequenza di bit risultante che immette:

$$\begin{cases} 1 : \#1 = \text{pari} \\ 0 : \text{altrimenti} \end{cases}$$

Si realizza prima un automa, in questo caso di **Moore**, ovvero che l'output è associato allo stato:

- S0: "1" Dispari
- S1: "1" pari

$$\begin{cases} S0 \to 0 \implies S0 \\ S0 \to 1 \implies S1 \\ S1 \to 0 \implies S1 \\ S1 \to 1 \implies S0 \end{cases}$$

Ecco il diagramma dell'automa:

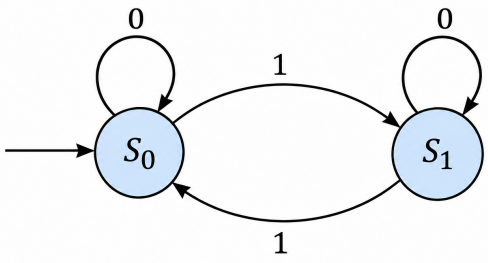


Figure 22: Automa

La tabella di verità è quindi:

Stato attuale (S)	Input(I)	Stato successivo (S^I)
0	0	0
0	1	1
1	0	1
1	1	0

Vediamo ora l'espressione logica associata:

$$S^I = \overline{S}I + S\overline{I}$$

Facciamo lo stesso per gli output:

Stato attuale (S)	Output(O)
0	1
1	0

$$O = \overline{S}$$

Se dovessimo realizzare un circuito che fa questo, ecco cosa dovremmo creare:

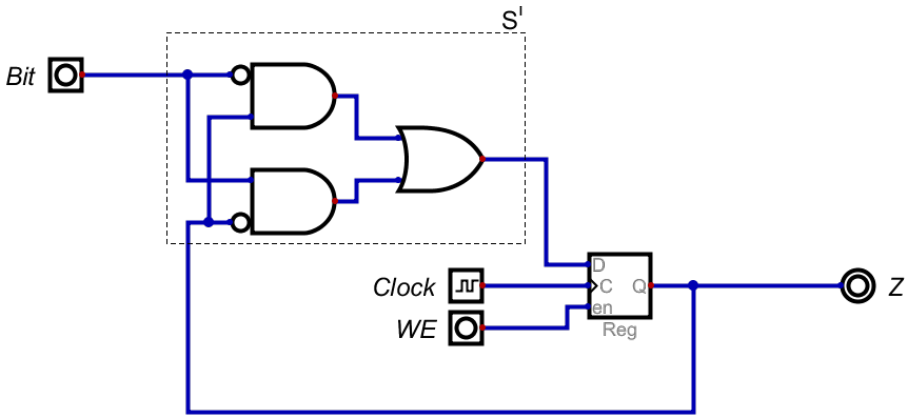


Figure 23: Circuito digitale dell'automa

I tempi legati al circuito sono:

- $S^I : 2\Delta t$
- $Z : 0$

Il tempo totale è quindi $2\Delta t$.
Riassumendo, per arrivare a capire come sviluppare una rete sequenziale tale, bisogna:

1. Capire il problema
2. Modellare un automa (Mealy o Moore)
3. Creare le tabelle di verità
4. Scrivere l'espressione logica annessa
5. Modellare il circuito digitale
6. Calcolare i tempi

7.7 RAM

La **RAM** (Random Access Memory) è una memoria che può essere scritta e letta.
Mantiene i dati sono in presenza di alimentazione.
Ha:

- **WE** in input.
- Numero in input (da n bit).
- Un indirizzo in input (da k bit), seleziona una tra le 2^k poisizioni.
- Un output che può essere:

$$out = \begin{cases} in, WE == 1 \\ 0, WE == 0 \end{cases}$$

Se il WE è settato a 1, allora l'indirizzo di memoria viene aggiornato con l'input immesso e si restituisce in uscita, altrimenti si mantiene il dato precedente.

$$M[ind] == In$$

$$out = M[ind]$$

Vediamo per esempio un circuito di memoria, ovvero il **Register File** (banco di registri) che sceglie quale registro utilizzare in base all'indirizzo.

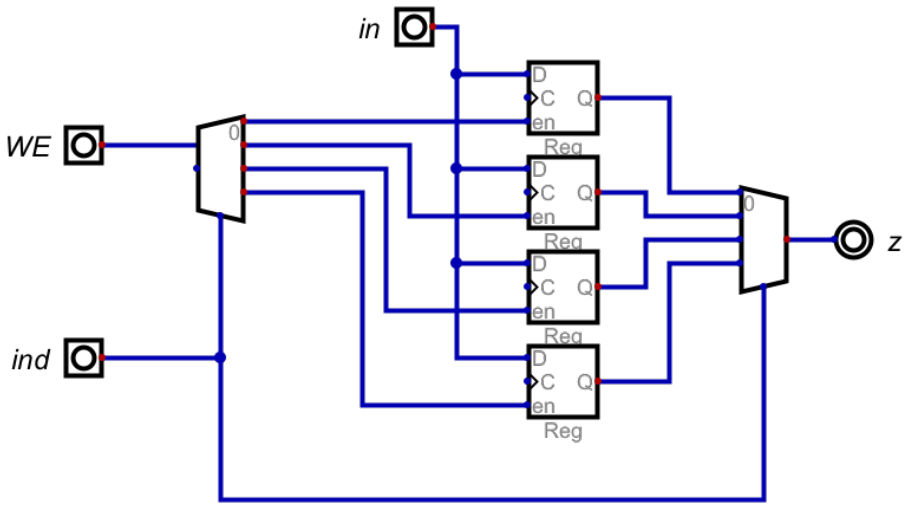


Figure 24: Register File

7.7.1 RAM Dinamica

In una RAM dinamica, un valore deve essere riletto periodicamente perche piano piano perde valore e potrebbe cancellarsi.

- Il valore "1" dopo un po' inizia a perdere potenza (elettrica) e si avvicina allo 0, fino a che non supera la soglia che distingue i due valori, diventando ambiguo.
- Lo stesso succede per lo "0", che piano piano si "sporca" se non riletto periodicamente.

Per risolvere, si introduce un'operazione di **Refresh** che sistema i valori.
Una RAM dinamica costa poco, però ha un tempo di accesso elevato.

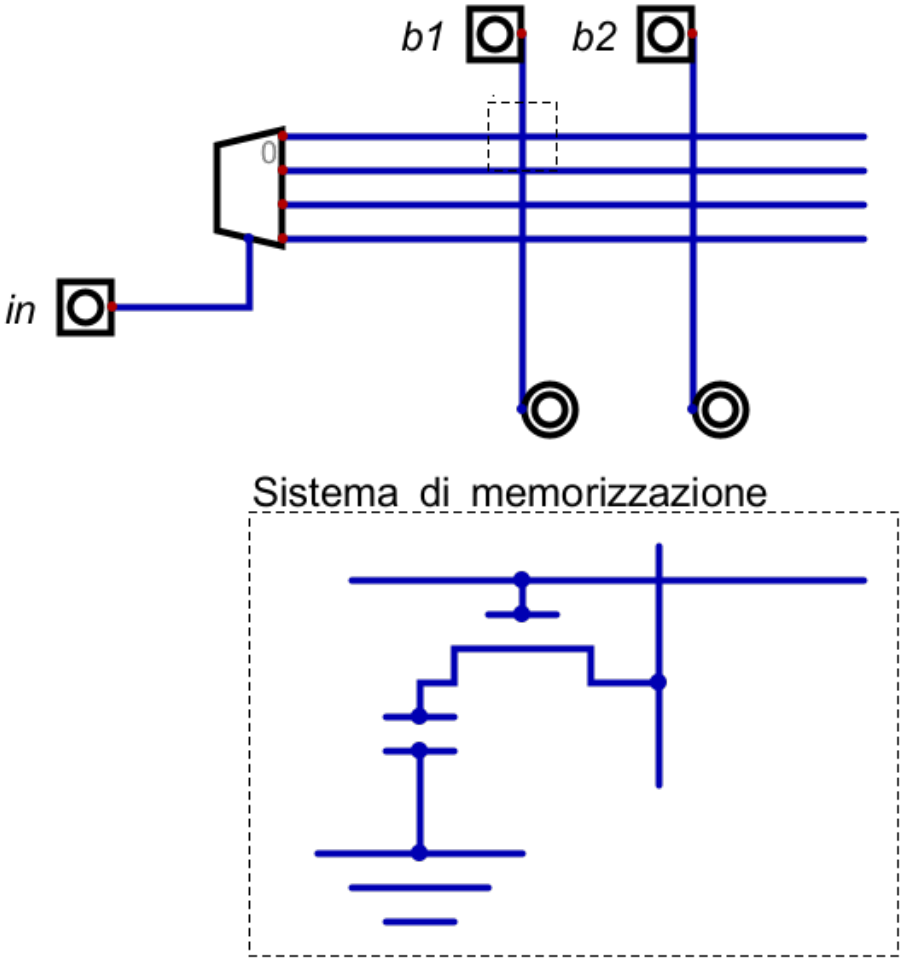


Figure 25: Ram dinamica

Il **sistema di memorizzazione** funziona in questo modo:

- Nel cavo di una porta aperta ("1") viene aperto un gate (transistor).
- Il gate fa andare la corrente verso la **terra**.
- La corrente incontra un **condensatore**, il quale conserva la carica elettrica (e quindi anche il valore).
- Quando viene immesso un altro "1", il gate si richiude e la corrente salvata nel condensatore viene emessa verso l'uscita (o la sua assenza per il valore 0).

Ovviamente il condensatore non tiene la carica per sempre ed è quindi per questo che i valori iniziano a sporcarsi dopo un po'.

7.7.2 RAM Statica

La RAM statica garantisce, finchè c'è alimentazione, la persistenza di un valore in una cella (non si sporca).

Una RAM del genere costa molto, però almeno garantisce che i valori rimangano puliti ed ha un tempo di accesso basso.

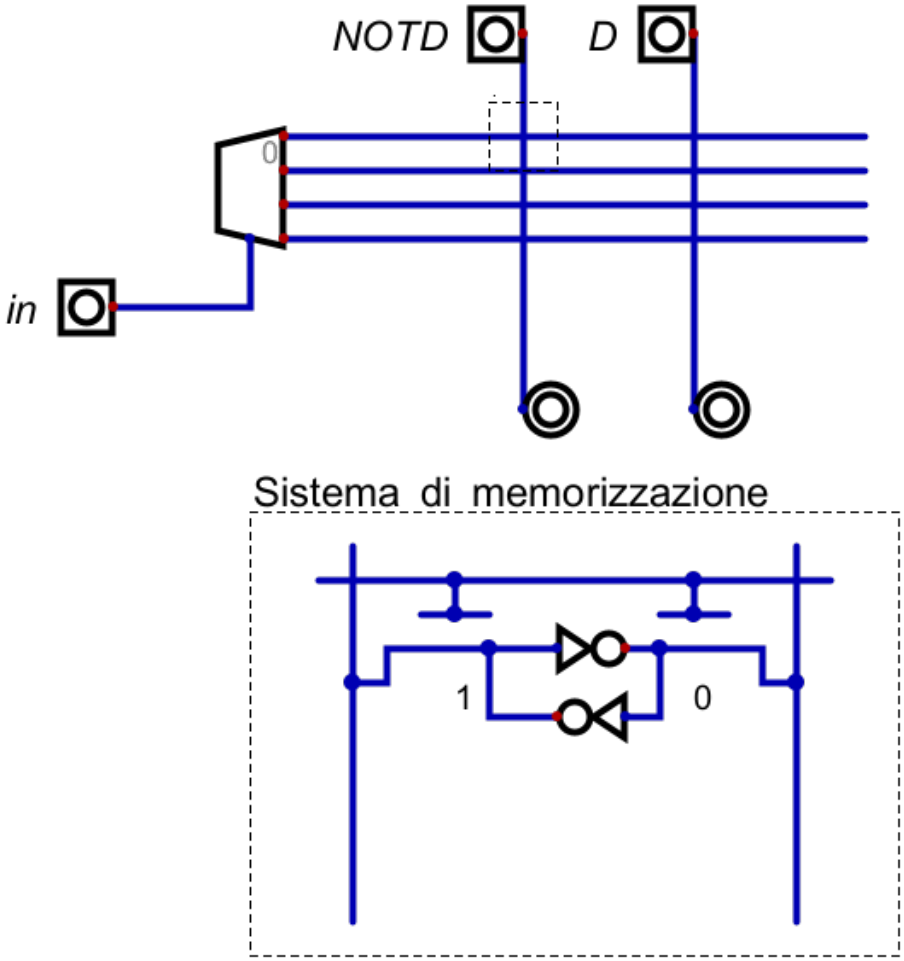


Figure 26: Ram statica

Il funzionamento è simile a quello della RAM dinamica, ci sono dei gate che vengono aperti e chiusi, ma il valore è memorizzato in dei **latch** che memorizzano anche il suo negato.
Il valore come si può vedere viene sempre ricalcolato dal NOT e non è salvato in un condensatore, quindi rimane stabile.

7.8 ROM

La **ROM** (Ready Only Memory), il vettore di valori è fisso, fatto in fabbrica, non si può modificare.
La ROM, al contrario della RAM, mantiene le informazioni anche in assenza di alimentazione.

Programmable ROM: modello di ROM che può essere scritta poche volte (come Flash USB).

7.9 Memorie associative

Esistono delle memorie che associano **chiavi** a **valori** (k, v).
Ha:

- WE in input.
- Valore in input (se si vuole scrivere)
- Chiave in input
- **HIT** in output (1 se si trova la chiave specificata)
- Valore trovato in uscita

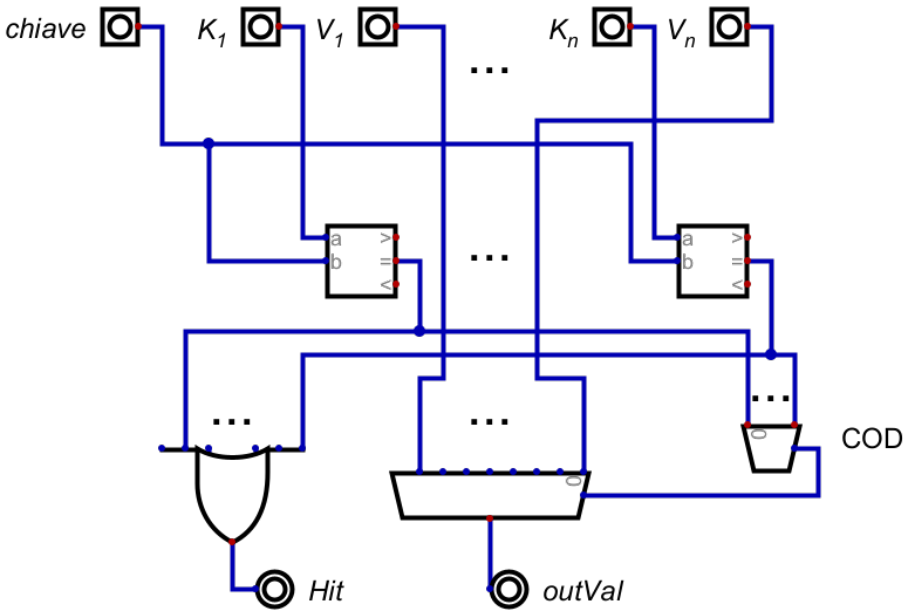


Figure 27: Memoria associativa

7.10 Memorie modulari

Spesso si compongono moduli di memoria più piccoli per crearne una composta, più grande, per avere dei vantaggi:

- Maggiore memoria
- Facile manutenzione
- Scalabilità

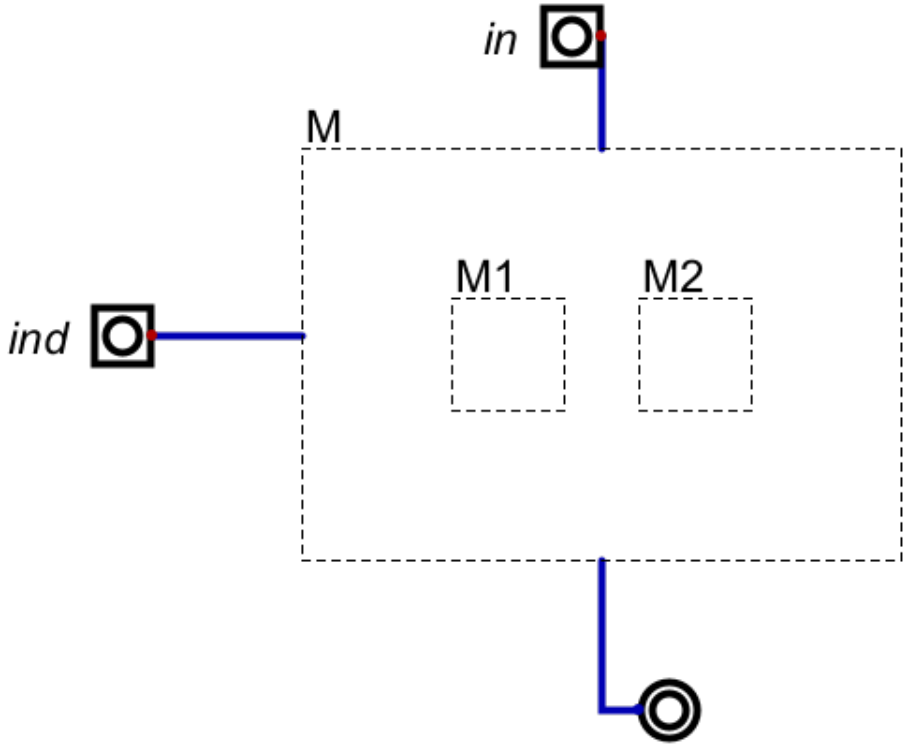


Figure 28: Memoria modulare d'esempio

L'indirizzo come sappiamo sceglie un indirizzo fra molti da utilizzare, i quali nelle memorie modulari sono suddivisi tra i moduli utilizzati.
La suddivisione degli indirizzi tra i vari moduli varia a seconda dell'**organizzazione** utilizzata.

7.10.1 Organizzazione sequenziale

Gli indirizzi sono suddivisi in modo da creare regioni contigue di indirizzi pari al numero dei moduli utilizzati.
Per esempio, se abbiamo 2048 indirizzi, questi saranno gli intervalli assegnati ai moduli:

- **M1:** 0 - 1023
- **M2:** 1024 - 2047

L'indirizzo sceglie quale modulo utilizzare tramite il MSB (o piu di 1 MSB se ci sono piu moduli) come in questo esempio (l'indirizzo in questo caso è da 11 bit):

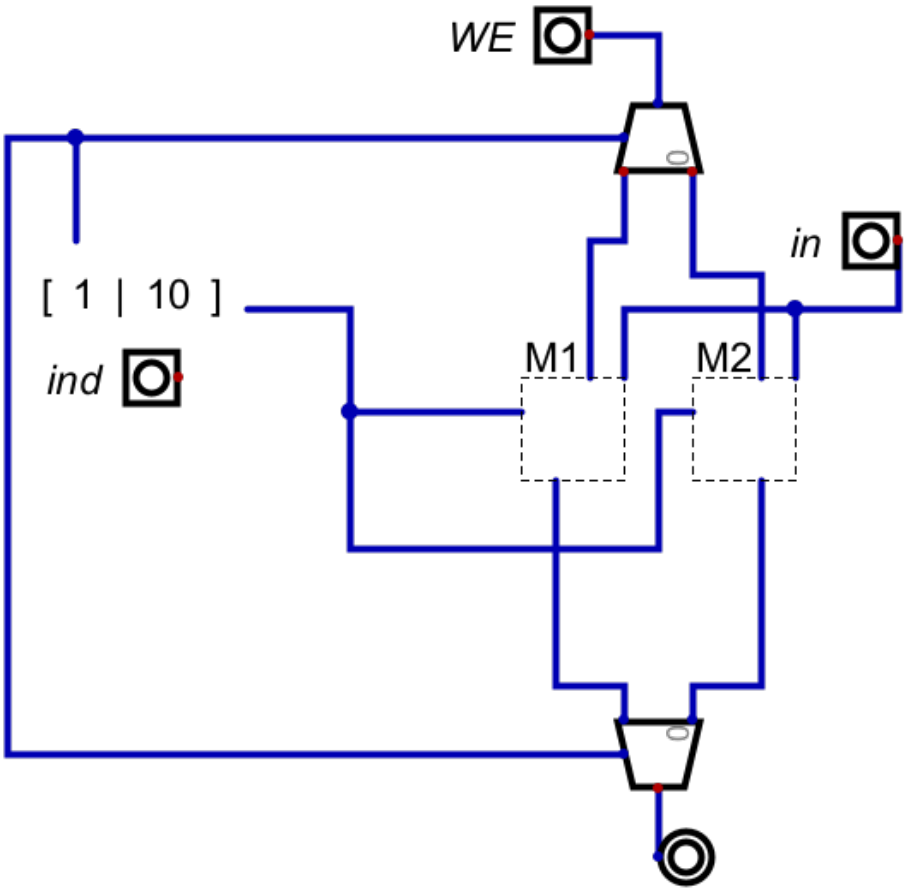


Figure 29: Memoria con organizzazione sequenziale

7.10.2 Organizzazione interlacciata

Con questo metodo, gli indirizzi sono suddivisi alternativamente, uno ad uno.
Si capisce subito che in questo modo un modulo ha gli indirizzi pari e l'altro quelli dispari, quindi la decisione spetta al LSB dell'indirizzo:

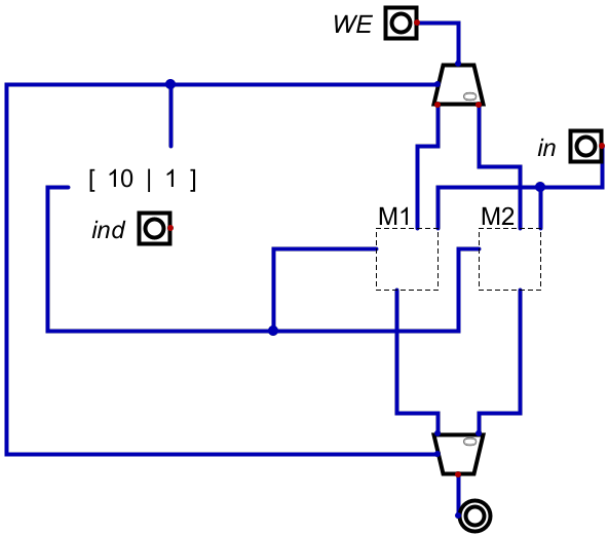


Figure 30: Organizzazione con memoria interlacciata

8 Assembler

8.1 Definizione assembler

L'**assembler** traduce il linguaggio **Assembly** in linguaggio macchina e consiste in un livello base di astrazione: le istruzioni fanno riferimento all'**architettura**, che può essere:

- x86
- ARMv7
- RISC-V

Noi andremo principalmente ad analizzare e utilizzare ARMv7, il quale utilizza 32 bit per registro per un totale di **16 registri**, andiamoli a vedere:

- **R0 - R12:** registri usati per memorizzare i dati necessari all'esecuzione.
- **R13 (Stack Pointer):** contiene l'indirizzo della cima dello stack.
- **R14 (Link Register):** contiene l'indirizzo a cui tornare una volta terminata una chiamata di funzione.
- **R15 (Program Counter):** contiene l'indirizzo dell'operazione da eseguire o della prossima.

8.2 PC e il suo valore

In particolare, il PC ogni volta che avanza non aumenta di 1, bensì di 4:

$$PC = PC + 4$$

Questo perche ogni istruzione corrisponde ad una **word**, ovvero 4 byte.
Questo varia però con le istruzioni dei salti:

$$PC = PC + [passi\ per\ il\ salto]$$

E per le guardie dei salti condizionali e aggiornamento della parola di stato:

$$PC = PC + 1$$

8.3 Parola di stato (CPSR)

La **parola di stato**, ovvero una stringa di 4 bit, è un registro speciale che contiene informazioni sul risultato delle operazioni e sullo stato del processore.
Nel caso di ARMv7, è chiamata **CPSR** (Current Program Status Register).
Ogni bit di questa word è un **flag** e ognuno indica una cosa diversa:

$$[\text{Z} — \text{N} — \text{C} — \text{V}]$$

- **Z:** 1 se il risultato è uguale a 0, 0 altrimenti.
- **N:** 1 se il risultato è negativo, 0 altrimenti.

- **C:** 1 se il risultato è uguale a 1, 0 altrimenti (carry finale).
- **V:** 1 se si è verificato overflow ($c \neq 0$), 0 altrimenti ($c = 0$).

Questo è molto utile per quando si fanno blocchi **condizionali**, in quanto si analizzano proprio questi flag per determinare una decisione.
In particolare:

- **N** e **Z** servono per condizioni logiche e aritmetiche.
- **C** serve solo per condizioni aritmetiche.
- **V** è a se' per l'overflow.

8.4 Codice assembler (Assembly)

8.4.1 Introduzione

Per usare l'assembler e capirlo meglio, andremo a creare programmi in **Assembly**, il quale viene svolto sequenzialmente dall'alto verso il basso, senza tornare indietro.
Anche questo codice necessita compilazione, viene tradotto in un altro file che contiene solo **operazioni binarie** (linguaggio macchina).

8.4.2 Schema interno

Ogni istruzione è salvata come una stringa binaria suddivisa in diversi frammenti che indicano gli argomenti, tra cui:

- Operatore (o funzione)
- Registri parametro

Vediamo un esempio con $ADD\ R_0, R_1, R_2$:

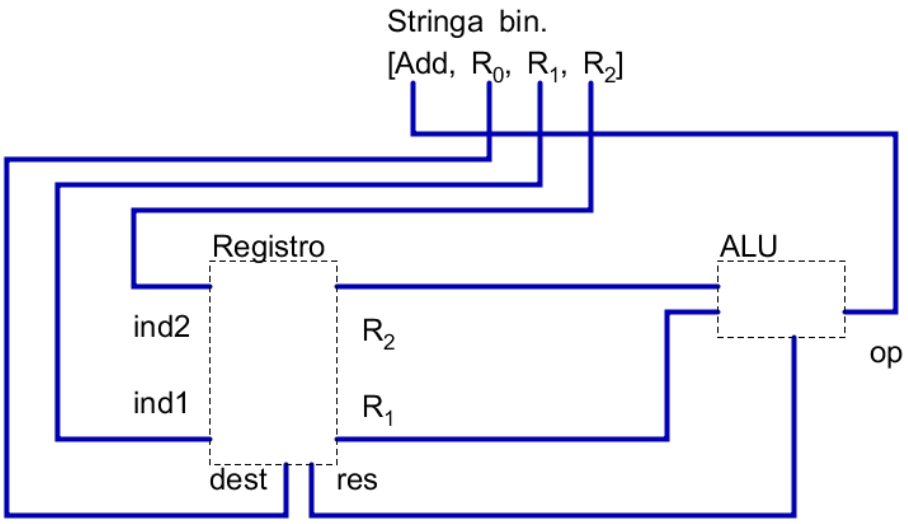


Figure 31: Assembler

8.4.3 Convenzioni nell'utilizzo dei registri

In Assembly esistono delle convenzioni per come bisogna utilizzare i diversi registri:

1. **R0 - R3** sono registri temporanei, ovvero registri in cui non serve mantenere i valori, mentre **R4 - R15** sono registri non temporanei, i valori vanno mantenuti.
2. I parametri attuali delle operazioni vanno salvati in **R0 - R3**
3. I valori di ritorno si salvano in **R0**
4. I valori non temporanei vanno mantenuti (ritrovati alla fine)

8.4.4 Sommario dei comandi

Ecco le istruzioni principali:

- Operative
  - Aritmetiche
    - ADD
    - SUB
    - MUL
    - DIV
  - Logiche
    - AND
    - OR
    - NOT
    - XOR
  - Shift / rotazione
    - SHIFT
    - ROT
  - ...
- Di Memoria
  - LOAD
    - Restituisce un valore da un indirizzo:

      LOAD val , addr
  - STORE
    - Salva un valore in un indirizzo:

      STORE val , addr
- di Salto
  - "Saltano" pezzi di codice, volendo anche secondo istruzioni, aggiornando il PC alla corrispondente istruzione.

```
inizio:
\dots
B fine:
-skipped code-
fine:
```

8.4.5 ADD

Il comando ADD svolge la somma tra due valori e lo salva in un registro terziario, richiede 3 parametri:

```
ADD [destinazione], [addendo1], [addendo2]
ADD R0, R1, R2
```

8.4.6 LSR e LSL

Questi comandi eseguono una **Logical Shift**, rispettivamente a destra (**Right**) o a sinistra (**Left**).
Ecco la loro struttura:

```
LSR [dest], [src], [quantiBit]
LSL [dest], [src], [quantiBit]

LSR R0, R1, #1
LSL R0, R2, #3
```

8.4.7 CMP

Questo comando va utilizzato per effettuare il confronto tra due registri prima di controllare una guardia:

```
CMP [registro1], [registro2]
CMP R0, R1
```

Se viene omesso, il salto condizionato fallisce

8.4.8 Salti

I salti generici sono denotati dalla lettera "B", permettono di saltare pezzi di codice per riprendere da un' **etichetta**:

```
...
B [etichetta]
B fine
...
fine:
```

Esistono anche salti che agiscono in base ad una condizione:

$$cmp\ [reg1], [reg2]$$

$$B[condizione]\ [etichetta]$$

- **BLT**: $reg1 < reg2$
- **BGT**: $reg1 > reg2$
- **BEQ**: $reg1 == reg2$
- **BLE**: $reg1 \leq reg2$
- **BGE**: $reg1 \geq reg2$
- **BNE**: $reg1 \neq reg2$

Esiste anche un altro tipo di salto che permette di simulare l'utilizzo di **funzioni**, salvando l'indirizzo di ritorno in LR:

```
BL [etichetta]
```

Il valore di ritorno è salvato nel registro **LR**, quindi per tornare indietro bisogna settare **PC** a **LR**:

```
MOV PC, LR
```

BL può essere combinato con una guardia se vogliamo.

8.4.9 LDR

Tale comando carica un valore dalla memoria del registro e lo salva in un altro:

```
LDR [dest], [[src]]
LDR R0, [R1]
```

Ci sono delle varianti:

```
LDR R0, =100 @ Carica un valore costante (pseudo istruzione)

LDR R0, [R1, #4] @ Carica R1 + 4

LDR R0, [R1], #4 @ Legge da R1, poi lo incrementa di 4

LDR R0, [R1, #4]! @ R1 += 4, poi legge dal nuovo indirizzo
```

8.4.10 Loop

Per realizzare loop bisogna utilizzare un salto all'indietro e un CMP appena si entra nel loop per simulare la guardia:

```
loop:   cmp R1,R2
        be fine:
        ...
        b loop

fine:
        ...
```

8.4.11 Operazioni condizionali

Esistono versioni di comandi base che collaborano con delle condizioni:

```
ADDEQ R1, R2, R3
```

Questo svolge $R2 - R3$, se il flag **Z** è 1 allora sono uguali e la condizione è vera e svolge quindi l'operazione.
Questo tipo di comandi permette di evitare l'utilizzo di salti, i quali possono provocare rallentamenti in codici eseguiti con pipeline, in quanto il programma si era preparato a svolgere certe operazioni e invece abbiamo deciso di saltarle.

Code	Suffix	Flags	Meaning
0000	EQ	Z set	equal
0001	NE	Z clear	not equal
0010	CS	C set	unsigned higher or same
0011	CC	C clear	unsigned lower
0100	MI	N set	negative
0101	PL	N clear	positive or zero
0110	VS	V set	overflow
0111	VC	V clear	no overflow
1000	HI	C set and Z clear	unsigned higher
1001	LS	C clear or Z set	unsigned lower or same
1010	GE	N equals V	greater or equal
1011	LT	N not equal to V	less than
1100	GT	Z clear AND (N equals V)	greater than
1101	LE	Z set OR (N not equal to V)	less than or equal
1110	AL	(ignored)	always

Figure 32: Mnemonics

8.4.12 Comandi che settano flag

Per fare in modo che dopo un'operazione lo stato del risultato venga mantenuto nei flag bisogna aggiungere una "S":

```
ADDS R0, R1, R2
```

Questa operazione esegue l'operazione base normalmente e poi setta i flag in base al risultato dell'operazione associata.
I flag non vengono normalmente settati, ci vogliono questi comandi.

8.4.13 Prelevare registri dallo stack

Se abbiamo bisogno di piu registri di quelli temporanei, bisogna utilizzare quelli seguenti, ma per farlo bisogna prima toglierli dallo stack e poi, una volta utilizzati, vanno rimessi nello stack:

```
PUSH {R4, R5}
...
POP {R4, R5}
```

Qui si salvano i valori di R4 e R5 per operare su di essi con altri valori, poi alla fine si riprendono i valori iniziali (si prendono i valori e si salvano nei registri specificati come parametri).
Questo è molto importante per realizzare delle **funzioni** in assembly:

```
BL func

func:
    PUSH {LR}
    ...
    POP {PC}
```

In questo codice, si chiama la funzione salvandoci l'indirizzo di ritorno che viene poi salvato nello stack. Man mano il PC aumenta e siccome ci siamo salvati l'indirizzo di ritorno prima, lo estraiamo di nuovo e lo salviamo in PC, in modo da tornare nel programma principale.

9 Altro

Rete combinatoria: il comportamento dipende solo dagli ingressi attuali.

Rete sequenziale: memorizza dati e il comportamento è influenzato dallo stato precedente.

10 Esercizi

10.1 Cosa calcola una rete

10.1.1 Descrizione

Dato una rete:

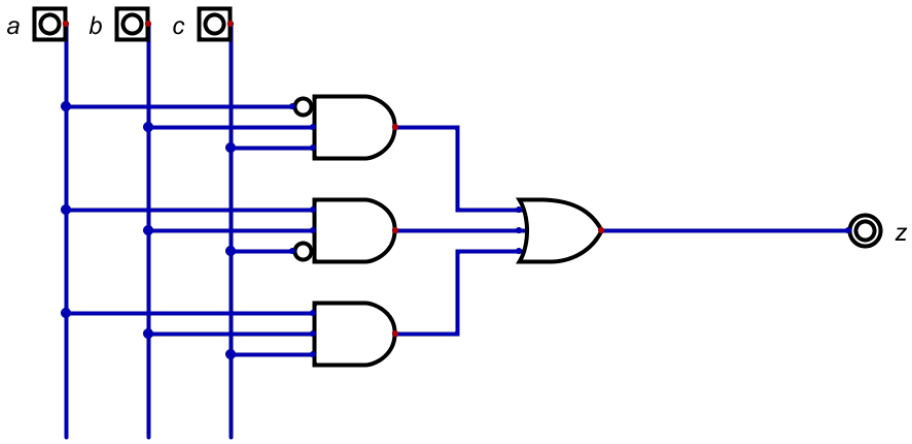


Figure 33: Rete sequenziale — Esercizio 1

Trovare:

1. Cosa calcola la rete.
2. Se è possibile un modo più efficiente di calcolarla.

10.1.2 Soluzione

Per trovare cosa calcola la rete, bisogna trovare l'**espressione** logica:

$$z = \overline{a}bc + ab\overline{c} + abc$$

Per capire invece se si può semplificare, bisogna provare in diversi modi:

Algebra booleana : si procede con l'effettuare raccoglimenti e semplificazioni.
In questo caso, si può aggiungere un quarto termine "abc" perche fare l'OR tra due valori uguali non cambia il risultato.

$$z = \overline{a}bc + ab\overline{c} + abc + abc$$

Adesso possiamo raccogliere:

$$z = (\overline{a} + a)bc + (\overline{c} + c)ab$$

$$z = bc + ab$$

$$(a + c)b$$

Come si può vedere si è semplificato molto.

Mappe di Kernaugh : si crea la tabella associata:

$a\backslash bc$	00	01	11	10
0	0	0	1	0
1	0	0	1	1

Si calcola l'espressione tramite i gruppi presi:

$$z = bc + ab = (a + c)b$$

10.2 Circuito di una sveglia

10.2.1 Descrizione

Un orologio digitale rappresenta le cifre necessarie per segnare il tempo con un totale di 7 segmenti che possono essere accesi o spenti a seconda della cifra rappresentata, di solito si nominano in questo ordine:

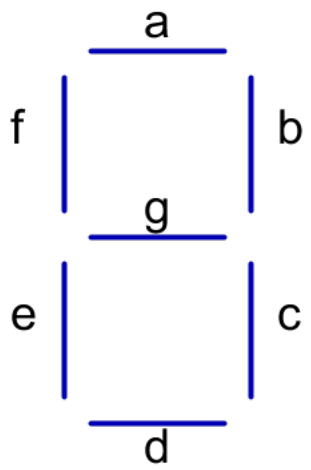


Figure 34: Segmenti orologio

Trovare l'espressione logica associata ai segmenti dell'orologio.

10.2.2 Soluzione

Iniziamo col dire che le cifre vanno da 0 a 9 e per rappresentare questi numeri in binario ci servono 4 bit.
Poi diciamo che ogni segmento (lettera):

$$segmento = \begin{cases} 1, & \text{segmento utilizzato} \\ 0, & \text{altrimenti} \end{cases}$$

Adesso si svolge la tabella di verità, la quale è molto grande:

Cifra	x_3	x_2	x_1	x_0	a	b	c	d	e	f	g
0	0	0	0	0	1	1	1	1	1	1	0
1	0	0	0	1	0	1	1	0	0	0	0
2	0	0	1	0	1	1	0	1	1	0	1
3	0	0	1	1	1	1	1	1	0	0	1
4	0	1	0	0	0	1	1	0	0	1	1
5	0	1	0	1	1	0	1	1	0	1	1
6	0	1	1	0	1	0	1	1	1	1	1
7	0	1	1	1	1	1	1	0	0	0	0
8	1	0	0	0	1	1	1	1	1	1	1
9	1	0	0	1	1	1	1	1	0	1	1
10	1	0	1	0	X	X	X	X	X	X	X
11	1	0	1	1	X	X	X	X	X	X	X
12	1	1	0	0	X	X	X	X	X	X	X
13	1	1	0	1	X	X	X	X	X	X	X
14	1	1	1	0	X	X	X	X	X	X	X
15	1	1	1	1	X	X	X	X	X	X	X

Come si può vedere molti casi sono nulli, infatti andrebbe fatto in modo che il circuito non accetti input con risultati nulli.

Per calcolare l'espressione (di ogni output(segmento)), si fa come al solito:

$$
\begin{aligned}
a = {} & \overline{x_3}\,\overline{x_2}\,\overline{x_1}\,\overline{x_0} + \overline{x_3}\,\overline{x_2}\,x_1\,\overline{x_0} \\
& + \overline{x_3}\,\overline{x_2}\,x_1\,x_0 + \overline{x_3}\,x_2\,\overline{x_1}\,x_0 \\
& + \overline{x_3}\,x_2\,x_1\,\overline{x_0} + \overline{x_3}\,x_2\,x_1\,x_0 \\
& + x_3\,\overline{x_2}\,\overline{x_1}\,\overline{x_0} + x_3\,\overline{x_2}\,\overline{x_1}\,x_0 \\
b = {} & \overline{x_3}\,\overline{x_2}\,\overline{x_1}\,\overline{x_0} + \overline{x_3}\,\overline{x_2}\,\overline{x_1}\,x_0 \\
& + \overline{x_3}\,\overline{x_2}\,x_1\,\overline{x_0} + \overline{x_3}\,\overline{x_2}\,x_1\,x_0 \\
& + \overline{x_3}\,x_2\,\overline{x_1}\,\overline{x_0} + \overline{x_3}\,x_2\,x_1\,x_0 \\
& + x_3\,\overline{x_2}\,\overline{x_1}\,\overline{x_0} + x_3\,\overline{x_2}\,\overline{x_1}\,x_0 \\
c = {} & \overline{x_3}\,\overline{x_2}\,\overline{x_1}\,\overline{x_0} + \overline{x_3}\,\overline{x_2}\,\overline{x_1}\,x_0 \\
& + \overline{x_3}\,\overline{x_2}\,x_1\,x_0 + \overline{x_3}\,x_2\,\overline{x_1}\,\overline{x_0} \\
& + \overline{x_3}\,x_2\,\overline{x_1}\,x_0 + \overline{x_3}\,x_2\,x_1\,\overline{x_0} \\
& + \overline{x_3}\,x_2\,x_1\,x_0 + x_3\,\overline{x_2}\,\overline{x_1}\,\overline{x_0} \\
& + x_3\,\overline{x_2}\,\overline{x_1}\,x_0 \\
d = {} & \overline{x_3}\,\overline{x_2}\,\overline{x_1}\,\overline{x_0} + \overline{x_3}\,\overline{x_2}\,x_1\,\overline{x_0} \\
& + \overline{x_3}\,\overline{x_2}\,x_1\,x_0 + \overline{x_3}\,x_2\,x_1\,x_0 \\
& + \overline{x_3}\,x_2\,\overline{x_1}\,\overline{x_0} + \overline{x_3}\,x_2\,x_1\,\overline{x_0} \\
& + x_3\,\overline{x_2}\,\overline{x_1}\,\overline{x_0} + x_3\,\overline{x_2}\,\overline{x_1}\,x_0 \\
e = {} & \overline{x_3}\,\overline{x_2}\,\overline{x_1}\,\overline{x_0} + \overline{x_3}\,\overline{x_2}\,x_1\,\overline{x_0} \\
& + \overline{x_3}\,x_2\,x_1\,\overline{x_0} + x_3\,\overline{x_2}\,\overline{x_1}\,\overline{x_0} \\
f = {} & \overline{x_3}\,\overline{x_2}\,\overline{x_1}\,\overline{x_0} + \overline{x_3}\,x_2\,\overline{x_1}\,\overline{x_0} \\
& + \overline{x_3}\,x_2\,x_1\,\overline{x_0} + x_3\,\overline{x_2}\,\overline{x_1}\,\overline{x_0} \\
& + x_3\,\overline{x_2}\,\overline{x_1}\,x_0 \\
g = {} & \overline{x_3}\,\overline{x_2}\,x_1\,\overline{x_0} + \overline{x_3}\,\overline{x_2}\,x_1\,x_0 \\
& + \overline{x_3}\,x_2\,\overline{x_1}\,\overline{x_0} + \overline{x_3}\,x_2\,\overline{x_1}\,x_0 \\
& + \overline{x_3}\,x_2\,x_1\,\overline{x_0} + x_3\,\overline{x_2}\,\overline{x_1}\,\overline{x_0} \\
& + x_3\,\overline{x_2}\,\overline{x_1}\,x_0
\end{aligned}
$$

L'esercizio è simile al precedente, ovviamente più ampio e complesso.